

Synthetic Market Phase 22

Path validation for the Leader axis using an early lesion at L4 and a downstream rescue at L40. The comparison is lesion only versus lesion plus rescue, both run on the compiled non-eager request-scoped patch path.

DATE	ROWS	PATCH	ENGINE
3 April 2026	84	L4 project_out + L40 swap_components	compiled non-eager custom-op

MAIN READ

Phase 22 does not give a clean action-choice path-validation result. The patching mechanics are clean, but the tool surface is too unstable at full scale: only 3 of 84 rows reached parsed tool calls in lesion, rescue, and source generation. On the full metric table rescue looks worse than lesion, but that headline is dominated by rows that never reached a comparable tool call.

TOOL-TOKEN RESTORE	TOOL-TOKEN BACKFIRE	SPEND IMPROVEMENT	LENGTH RESTORATION
0.133	0.571	n/a	-150.375
2 of 15 restorable rows	lower is better on rows already matching the source	share of spend-mismatched rows that moved closer to the source	negative means the rescue moved farther from the source length

What Was Tested

Phase 22 is a path-validation test, not a plain ablation test.

For each matched pair, the experiment generated:

- the source prompt on its own
- the base prompt with the early lesion only
- the base prompt with the same lesion plus a downstream rescue

The practical question is:

- after damaging the early Leader signal, can a downstream rescue restore source-like behavior?

In this report:

- the *baseline* is the lesion-only condition
- the *intervention* is lesion plus rescue

That matters because a positive result would mean:

- the later rescue site still carries enough of the causal path to recover behavior after the earlier lesion

Methodology In Plain English

`pair` mode here is just how matched examples are defined.

For `denoise`:

- the base row is the lower-valued member of the pair on `#sample.at("pair_metric")`
- the source row is the higher-valued member

`project_out` means:

- remove the selected Leader coefficients at the early layer

`swap_components` means:

- capture the source row
- average over the market span
- take the selected source-side coefficients
- write those coefficients into the base row at the rescue layer

So this is a specific directional test:

- break the early signal
- then try to restore the later signal

Tool Surface Validity

Measure	Count
Lesion rows with parsed tool calls	22
Rescue rows with parsed tool calls	22
Source rows with parsed tool calls	22
Rows with parsed tool calls in all three conditions	3
Source rows that hit <code>max_tokens = 15000</code>	62

This is the key caveat for Phase 22:

- the patch path itself worked
- but the full-scale behavior surface did *not* reliably reach parsed tool calls
- that means the action-choice metrics are only weakly interpretable at this scale

Lesion Versus Rescue

Measure	Lesion → Rescue	How to read it
Source tool-name match	0.409 → 0.227	Did the rescue move the chosen tool toward the source tool?
Source tool-token match	0.318 → 0.227	Did the rescue move the chosen token toward the source token?
Tool-token restoration rate	0.133	Higher is better on rows that needed fixing
Tool-token backfire rate	0.571	Lower is better on rows that were already correct
Mean source spend gap	0 → 0	Lower is better
Spend improvement rate	n/a	Share of spend-mismatched rows that moved closer
Mean source length gap	4348.25 → 5674	Lower is better
Mean normalized length restoration	-150.375	Negative means the rescue moved farther away

Restorable Rows And Backfires

Metric	Counts
Tool name	restorable 13 restored 2 backfires 6 / 9
Tool token	restorable 15 restored 2 backfires 4 / 7
Spend pct	restorable 0 improved 0 backfires 0 / 3
Generated tokens	restorable 35 improved 13 backfires 19 / 84

This is the most useful operational readout *when the tool surface is valid*:

- restoration asks whether rescue helps on rows that the lesion got wrong
- backfire asks whether rescue hurts rows that the lesion already had right

Mechanical Validity

Measure	Value
Rows with patch stats	84
Patch applied rate	1
Patch skipped rate	0

Mechanically, this section answers whether the intervention path actually fired. A positive path-validation result only matters if the rescue was really applied.

Methodology Scorecard

Best-practice item	Status
Matched prompt pairs and separately generated source behaviors	meets
Early lesion plus downstream rescue in the same request-scoped engine path	meets
Compiled non-eager custom-op execution with batch size 32	meets
Batched donor capture on Modal rather than one-by-one residual capture	meets
Per-row patch diagnostics with skip accounting	meets
Stable parsed tool-call surface across lesion, rescue, and source runs	missing
Downstream rescue gives cleaner action restoration than lesion alone	missing
Response length / verbosity also restores cleanly	missing
Further downstream path tracing beyond one rescue site	missing

The main methodological upgrade over Phase 21 is that this is not only a restoration test. It is a *path validation* test: early lesion first, downstream rescue second.

Interpretation

The most important question is not whether the rescue changes behavior at all. The question is whether it changes behavior in the *source-consistent direction* after the lesion has already damaged the early signal.

So the strongest positive pattern would be:

- tool-token restoration meaningfully above zero
- tool-token backfire clearly lower than restoration
- spend moving closer to source
- patch application mechanically clean

But before reading those metrics causally, one precondition has to hold:

- lesion, rescue, and source all have to reach comparable parsed tool calls often enough to make the action surface observable

The weakest pattern would be:

- restoration near zero
- backfire comparable to or larger than restoration
- no improvement in spend or source-match metrics

If the tool surface is sparse or truncated, the safer interpretation is narrower:

- the current full-scale setup is not yet a valid path-validation benchmark for action choice
- the patching mechanics can still be correct while the readout is not strong enough